

A Study of Asynchronous Weight-Updating Federated Learning for IoT Health Devices

Final Report – Undergraduate Thesis

Thomas Llamzon

Department of Computer Science, Western University

March 2026

Project Supervisor: Zubair Fadlullah, PhD, Department of Computer Science

Course Coordinator: Prof. Nazim Madhavji, Dept. of Computer Science

Glossary

FL	Federated Learning – a distributed machine learning paradigm where multiple clients collaboratively train a shared model without exchanging raw data.
CNN	Convolutional Neural Network – a class of neural networks commonly used for structured grid data such as images and time-series signals.
ECG	Electrocardiogram – a recording of the electrical activity of the heart over time using electrodes placed on the body.
FedAvg	Federated Averaging – the canonical FL aggregation algorithm where the server computes a weighted average of client model parameters [1].
IID	Independent and Identically Distributed – a data partitioning assumption where each client’s local dataset is drawn from the same underlying distribution.
Non-IID	Non-Independent and Identically Distributed – a more realistic data partitioning where client data distributions differ, often simulated using Dirichlet allocation.
IoT	Internet of Things – a network of physical devices embedded with sensors and software that connect and exchange data, often under bandwidth and compute constraints.
PTB-XL	A large, publicly available 12-lead ECG dataset from PhysioNet containing 21,799 clinical ECG records [2].
NCSS	Network Communication Scheduling Strategy – the policy governing when and which model parameters are transmitted between clients and server during FL training.
NORM	Normal sinus rhythm – the diagnostic superclass label in PTB-XL representing healthy heart activity.

Structured Abstract

Context: Federated Learning (FL) allows multiple devices to collaboratively train a shared model without centralizing raw data. However, clients in traditional, synchronous FL naively upload all model parameters in every training round, incurring an unnecessary amount of communication overhead. This overhead is especially problematic for Internet of Things (IoT) health devices which operate under network bandwidth and compute constraints.

Study Purpose: This thesis examines whether a periodic, layer-wise asynchronous update schedule can reduce communication overhead in FL while maintaining acceptable model utility on an ECG classification task under tightly controlled conditions.

Method: This study is designed as a controlled proof-of-concept simulation. A 1D Convolutional Neural Network (CNN) for binary classification (normal vs. abnormal) was trained on the PTB-XL ECG dataset across three architectures: centralized, synchronous FL, and asynchronous FL. The federated setup used 3 simulated clients, 4 communication rounds, and 1 participating client per round. The asynchronous strategy partitions model parameters into shallow and deep layers, transmitting shallow updates every round and deep updates every K rounds. All architectures used matched training budgets, identical model configurations, and the same dataset preprocessing pipeline to isolate the effect of the scheduling mechanism under IID and non-IID data partitions.

Results: In this proof-of-concept setting, asynchronous FL reduced total communication by 24.8% at $K = 2$ while reducing upload by 49.70%. At $K = 4$, upload reduction reached 74.55%. Reported accuracy values are single-run outcomes under a fixed seed; they are descriptive results for this setup, not evidence of general deployment performance. Across all async runs, accuracy improved only after full (deep) aggregation rounds.

Conclusions: Periodic layer-wise asynchronous updating reduced uplink communication while keeping the $K = 2$ run close to the synchronous baseline after 4 rounds. $K = 4$ achieved larger communication savings but weaker learning. These results support the scheduling mechanism under controlled minimal conditions, but they do not establish convergence or deployment-scale performance.

Contents

Glossary	i
Structured Abstract	ii
1 Introduction	1
General Background and Problem Description	1
Focus of This Thesis	1
Key Research Question Addressed	1
Key Results	1
Novelty and Significance	2
Report Structure	2
2 Background and Related Work	3
2.1 Federated Learning Basics	3
2.2 Asynchrony in Distributed Optimization	3
2.3 Communication-Efficient Federated Learning	3
2.4 IoT Health Device Constraints	4
2.5 Analysis and Research Gap	4
3 Research Goal and Objectives	5
4 Methodology	6
4.1 Research and Development Approach	6
4.2 Dataset	6
4.3 Model Architecture	6
4.4 Experimental Design	7
5 Results	8
5.1 System Overview and Validation	8
5.2 Centralized Baseline Results	9
5.3 Synchronous FL Results	9
5.3.1 IID Partition	9
5.3.2 Non-IID Partition	10
5.4 Asynchronous FL Results	11
5.4.1 Implementation Validation ($K = 1$, IID)	11
5.4.2 IID Experiments ($K = 2$ and $K = 4$)	12
5.4.3 Non-IID Experiments ($K = 2$ and $K = 4$)	13
5.4.4 Shallow-Round Stagnation	14
5.5 Communication Cost Comparison	14
5.6 Novelty of Results	15
6 Discussion	16
6.1 Threats to Validity	16
6.2 Implications of Research Results	16
6.3 Limitations of Results	17
6.4 Generalisability of Results	17

6.5	Value of the Controlled Small-Scale Setup	18
7	Conclusions	19
8	Future Work and Lessons Learnt	20
8.1	Future Work	20
8.2	Lessons Learnt	20
	Acknowledgements	21
	Appendix	22
	Appendix A: Tables	22
	Appendix B: Detailed Methodology	23
	References	27

List of Figures

5.1	System Context Diagram showing the three modules and their shared components. .	8
5.2	Training and validation loss and accuracy curves for the centralized ECG CNN across 4 epochs on PTB-XL (NORM vs. Abnormal).	9
5.3	Test loss and accuracy by round for the synchronous FL model (IID partition, 3 clients, 4 rounds, 1 local epoch per round).	10
5.4	Test loss and accuracy by round for the synchronous FL model (non-IID partition, Dirichlet $\alpha = 0.5$, 3 clients, 4 rounds, 1 local epoch per round).	10
5.5	Test loss and accuracy by round for the async FL model (IID, $K = 1$, validation run). Round types: all full.	11
5.6	Test loss and accuracy by round for the async FL model (IID, $K = 2$). Round types: shallow, full, shallow, full.	12
5.7	Test loss and accuracy by round for the async FL model (IID, $K = 4$). Round types: shallow, shallow, shallow, full.	12
5.8	Test loss and accuracy by round for the async FL model (non-IID, $K = 2$). Round types: shallow, full, shallow, full.	13
5.9	Test loss and accuracy by round for the async FL model (non-IID, $K = 4$). Round types: shallow, shallow, shallow, full.	13

List of Tables

8.1	Round-by-round test accuracy and loss for both synchronous FL experiments.	22
8.2	Round-by-round test accuracy for IID experiments. Asterisks (*) mark shallow-only rounds.	22
8.3	Round-by-round test accuracy for non-IID experiments. Asterisks (*) mark shallow-only rounds.	22
8.4	Communication cost and model utility across all federated experiments.	22
8.5	Notation for the asynchronous periodic deep-layer schedule and its correspondence to the implementation.	25

1 Introduction

General Background and Problem Description

Standard, synchronous federated learning (FL) networks suffer from high network communication overhead between clients and their server. The drawbacks of this are especially prevalent in resource-constrained devices such as IoT health devices that should be able to dedicate the majority of their resources to anomaly detection rather than local model updating. To optimize communication, this thesis proposes asynchronous weight-updating to mitigate the drawbacks of naive deep-layer parameter updating behaviour that synchronous FL suffers from.

Focus of This Thesis

This research focuses on evaluating whether asynchronous, layer-wise parameter updates can reduce network communication overhead in federated learning for IoT health device settings. The study does not aim to improve individual CNN model performance (e.g., reducing loss or increasing accuracy beyond what centralized training achieves), nor does it seek to discover the optimal asynchronous layer-wise parameter sending schedule. It is a controlled proof-of-concept simulation with 3 clients and 4 communication rounds, intended to isolate the scheduling mechanism under minimal conditions rather than to approximate a real deployment. The scope is confined to measuring whether the proposed schedule produces measurable communication savings while maintaining acceptable model utility in that limited setting. The design is intentionally narrow so causal attribution is clearer: with federation size, training budget, architecture, and preprocessing held constant, observed differences can be interpreted primarily as effects of update cadence rather than side effects of unrelated system changes.

Key Research Question Addressed

To what extent does the asynchronous weight-updating strategy reduce network communication scheduling overhead in federated learning under IoT health device constraints?

The experiments provide an initial, scoped answer for this controlled setting.

Key Results

In the 3-client, 4-round simulation used here, asynchronous weight-updating reduced client upload volume by 49.70% at $K = 2$ and 74.55% at $K = 4$ relative to synchronous FedAvg. These accuracy values are single-run results under a fixed seed and should be interpreted as descriptive outcomes of the controlled setup, not as evidence of general performance. Across all asynchronous experiments, model accuracy did not change during shallow-only rounds and improved only after rounds that included deep-parameter aggregation. In this setup, K therefore controls how many rounds include a deep update within the fixed budget.

Within this controlled setup, asynchronous weight-updating reduces federated-learning uplink communication cost under simulated IoT health device conditions.

Novelty and Significance

This work contributes the following:

1. A formalized, layer-aware (white-box) asynchronous shallow/deep weight update schedule for FL that separates model parameter transmission by layer depth.
2. Implementation and evaluation of this schedule on a PTB-XL-based ECG classification task representative of IoT health monitoring.
3. A controlled experimental comparison across centralized, synchronous FL, and asynchronous FL architectures with matched compute and data budgets.
4. Identification of a practical design consequence: when the shallow partition is extremely small, shallow-only rounds may contribute little measurable learning within a short training budget.

This contribution should be interpreted as methodological rather than definitive. The thesis introduces and stress-tests a reproducible scheduling mechanism in a minimal experimental harness designed for clear comparison, providing evidence and design hypotheses for larger-scale follow-up studies.

Report Structure

This thesis report first establishes the context and current research progress in the federated learning and asynchronous weight-updating field. Then, the research goals and objectives of this thesis are explicitly declared. This is followed by a high-level overview of the methodology used to conduct robust, defensible, and justified experiments. After the groundwork of the thesis setup is established, the results yielded from the experiments conducted are presented and interpreted. Afterwards, this thesis' threats to validity, limitations, implications, and generalizability are discussed, as well as the basis for future work the study provides. The thesis then closes with a conclusion followed by final remarks and citations.

2 Background and Related Work

2.1 Federated Learning Basics

Federated Learning (FL) trains a shared model across multiple clients by exchanging parameter updates rather than raw data [1]. In FedAvg, the server distributes the global model each round, clients train locally, and updated parameters are averaged on return [1]. Yang et al. [3] categorized FL into horizontal, vertical, and transfer learning variants; healthcare IoT falls under horizontal FL, where devices collect the same signal types from different patients. Their framework assumes synchronous rounds, homogeneous participation, and uniform parameter handling, and does not address how update timing or layer-specific schedules affect efficiency.

2.2 Asynchrony in Distributed Optimization

Standard FL requires the server to wait for all sampled clients before aggregating, which creates idle time when clients differ in hardware, availability, or connectivity [4]. Asynchronous variants relax this requirement by allowing updates to be incorporated without strict round-level synchronization. Hartmann [5] frames FL efficiency around two levers: reducing the amount of data sent per round and reducing the number of rounds needed to learn a useful model. Tariq et al. [6] similarly note that partial participation and periodic aggregation can make FL more tolerant of asynchronous device behavior.

Most prior work, however, still treats the model as a single parameter block. This thesis studies a narrower form of asynchrony: the model is partitioned into shallow and deep parameter groups, and the two groups are synchronized at different temporal frequencies. That design is attractive from a communication perspective, but it changes the optimization problem. During shallow-only rounds, the server updates only part of the model while keeping the deep component fixed. The resulting step is no longer the same full-model update assumed in standard FedAvg analyses, so client-server drift and update mismatch can grow even if communication is reduced. For that reason, layer-specific temporal decoupling is best treated here as a practical scheduling mechanism whose convergence properties remain analytically open.

2.3 Communication-Efficient Federated Learning

Communication is the primary bottleneck in FL because clients must download the full model and upload updates for all parameters each round, with costs growing proportionally to model size [5]. Upload bandwidth is typically 3 to 10 times slower than download, making upload compression especially important. Hartmann [5] proposes several compression techniques: sparse masks (randomly selecting a fraction p of parameters, achieving sublinear communication at $p = \log(m)/m$ for m parameters), probabilistic quantization (encoding updates in fewer bits while maintaining unbiased estimates), and structured updates (constraining updates to low-rank representations). These were validated at scale across 360,000 Mozilla Firefox users. Yuan et al. [7] take a different approach inspired by SplitNN [8]: they decompose the neural network so that a shallow sub-network runs on the IoT device while deeper layers run on a centralized server, then sparsify the activations and gradients exchanged between them (top 10% of elements). On a PhysioNet arrhythmia detection task, their framework reduced per-iteration traffic by 99.8% relative to FedAvg (1.36 GB down to 3.2 MB for 16 devices) with less than 2% accuracy loss. However, both Hartmann and Yuan et

al. treat parameter updates uniformly across layers and operate under synchronous assumptions. Neither evaluates layer-differentiated update frequencies as a communication reduction strategy.

2.4 IoT Health Device Constraints

IoT health devices operate under bandwidth, compute, power, and connectivity constraints that limit the feasibility of standard FL [7]. Wearable ECG monitors, pulse oximeters, and similar devices typically have limited processors, small batteries, and intermittent wireless connections. Yuan et al. [7] note that vanilla FL is insufficient for healthcare IoT because these devices cannot handle the full computation and communication load of standard FedAvg rounds. Tariq et al. [6] identify unreliable networks, limited throughput, dynamic bandwidth, and non-IID data distribution as factors that disrupt FL training in edge and IoT environments. They note that edge devices are well-suited for distributed training due to widespread deployment, but face participation constraints from resource limitations and intermittent connectivity. These constraints motivate communication strategies that reduce the volume and frequency of parameter transmission, rather than assuming reliable, high-bandwidth links between all clients and the server.

2.5 Analysis and Research Gap

Yang et al. [3] formalized FL’s architecture and defined accuracy-loss thresholds, but assumed synchronous full-model rounds with no consideration of layer-specific scheduling. Tariq et al. [6] identified communication efficiency as a prerequisite for trustworthy FL in constrained environments and catalogued relevant techniques, but did not implement or empirically evaluate any specific strategy. Yuan et al. [7] achieved 99.8% per-iteration traffic reduction vs. FedAvg with less than 2% accuracy loss by partitioning computation between device and server, but their approach assigns fixed layer roles and relies on synchronous rounds; it optimizes where computation runs, not when or how often each parameter group is communicated. Hartmann [5] demonstrated that sparse masks and quantization can compress FL uploads to sublinear cost and validated this across 360,000 users, but compression is applied uniformly across all layers without exploiting the fact that shallow and deep layers may converge at different rates.

Yuan et al. [7] showed that partitioning a network by layer depth reduces traffic, and Hartmann [5] showed that selectively omitting parameter updates preserves model quality at scale. This thesis combines both observations: rather than splitting computation across locations (Yuan et al.) or compressing all parameters uniformly (Hartmann), we partition model parameters into shallow and deep groups and transmit them at different frequencies within standard FedAvg rounds. Shallow layers, which learn general low-level features and converge relatively quickly, are sent every round. Deep layers, which learn slower-changing task-specific patterns, are sent every K rounds. This periodic asynchronous schedule is implemented and evaluated on the PTB-XL ECG dataset [9] under matched training budgets, identical model architectures, and both IID and non-IID distributions, with communication volume, convergence behavior, and model utility explicitly measured against centralized and synchronous FL baselines.

3 Research Goal and Objectives

The goal of this thesis is to investigate whether asynchronous, layer-wise weight-updating in federated learning can reduce network communication overhead while maintaining acceptable model utility for IoT health device ECG classification.

- **O1:** Implement and validate a centralized ECG CNN baseline on the PTB-XL dataset to establish reference performance metrics.
- **O2:** Implement synchronous FL training of the same CNN to establish baseline federated communication costs and model utility.
- **O3:** Design and implement an asynchronous FL strategy that partitions model parameters into shallow and deep layers and transmits them at different frequencies.
- **O4:** Evaluate the communication cost reduction and model utility trade-offs of the asynchronous strategy under IID and non-IID data distributions.

Together, these objectives establish rigorous baselines (centralized, synchronous FL), introduce and implement the proposed asynchronous layer-wise update schedule, and systematically evaluate its impact on communication cost and model utility. This directly addresses the thesis goal of quantifying the benefits and trade-offs of asynchronous, decoupled parameter updates for IoT ECG classification.

4 Methodology

This study uses an experimental comparison across three architectures (centralized, synchronous Federated Learning (FL), and asynchronous FL) to evaluate whether layer-wise asynchronous parameter updates reduce communication overhead while preserving model utility. All three architectures train the same 1D Convolutional Neural Network (CNN) on the PTB-XL electrocardiogram (ECG) dataset for binary classification (normal vs. abnormal heart readings), using identical preprocessing, hyperparameters, and training budgets to ensure fair comparison. Detailed variable definitions and module configurations are provided in Appendix C.

4.1 Research and Development Approach

The system was implemented in Python using PyTorch [10] for deep learning and Flower [11] for FL orchestration. The centralized module establishes a performance baseline. The synchronous FL module implements standard FedAvg [1] to establish a communication baseline. The asynchronous FL module introduces a periodic schedule that partitions model parameters into shallow and deep subsets: shallow parameters (early convolutional layers) are transmitted every round, while deep parameters (later convolutional layers and the classification head) are transmitted every K rounds. This partition exploits the observation that shallow layers learn general, low-level features that converge relatively quickly, while deep layers learn task-specific patterns that change more slowly during training. Unlike standard FedAvg, where the server treats the model as an opaque flat parameter vector, this approach requires **white-box orchestration**: the server must have explicit knowledge of the model’s internal layer structure (e.g., which parameter keys belong to `SHALLOW_PREFIXES` and which belong to the complement) so that clients and server agree on what is uploaded each round. The server is not aggregating an undifferentiated blob of weights; it is coordinating two distinct parameter groups on two different schedules.

4.2 Dataset

This project uses the PTB-XL dataset [2], a large publicly available 12-lead ECG dataset from PhysioNet containing 21,799 clinical records sampled at 500 Hz. Each sample is a 10-second recording with a tensor shape (5000, 12) representing time steps and leads. The dataset provides 10 stratified folds for cross-validation with balanced class distribution: 44% normal sinus rhythm (NORM) vs. 56% abnormal readings. Signals were normalized to zero mean and unit variance. Folds 1–9 were used for training, and Fold 10 was reserved as the held-out test set across all architectures.

4.3 Model Architecture

Both centralized and federated implementations use the same 1D CNN (ECGCNN). The model architecture consists of hierarchical convolutional layers: two initial shallow blocks (`Conv1d` + `BatchNorm` + `ReLU`) with smaller kernels that extract higher-frequency features such as waveform morphology, followed by deeper convolutional layers with larger kernels that capture complex rhythm patterns and temporal dependencies across heartbeats. A global average pooling layer aggregates temporal features into a fixed-size vector, processed by a dense classification head with dropout regularization (`DROPOUT_RATE=0.4`) to output a single diagnostic probability. The task is binary classification: Normal (NORM) vs. Abnormal.

4.4 Experimental Design

Experiments were conducted sequentially. First, the centralized CNN was trained to establish a model performance baseline. Then synchronous FL was run under IID data partitioning to establish a federated communication baseline. Finally, asynchronous FL was run with IID partitioning to test the core research question, followed by non-IID partitioning in further synchronous and asynchronous experiments. The federated setup was intentionally minimal: 3 simulated clients, 4 communication rounds, and 1 participating client per round.

This small-scale design is deliberate. It provides a controlled environment where schedule-induced communication effects are visible without being obscured by large-scale system variance (multi-node network jitter, broad hardware heterogeneity, and long-horizon drift). In this sense, the setup serves as an instrumentation-first proof of concept: the primary goal is clean measurement and mechanism isolation, not deployment realism.

All experiments used a fixed `RANDOM_SEED=42` for client data partitioning, weight initialization, and local training. No sensitivity analysis across multiple seeds was performed. This means the $K = 2$ accuracy results (83.08% IID, 84.62% non-IID) reflect a single random initialization, and it is not possible to determine from these results alone whether the observed accuracy is stable or a favourable outcome of this particular seed. A multi-seed replication would require re-running the full experiment set several times over, which was not feasible within the time constraints of this project. Seed sensitivity analysis is left as future work (Section 8).

A unified training configuration was applied across all architectures: Adam optimizer, `LEARNING_RATE=1e-3`, `BATCH_SIZE=32`, 4 epochs (centralized) or 4 rounds with 1 local epoch (federated), and 3 clients with 1 client participating per round. The asynchronous schedule was evaluated at $K \in \{1, 2, 4\}$. These values were chosen because they divide the fixed 4-round budget cleanly and create three interpretable cases: no deep staleness, one shallow-only round between deep updates, and a single deep update within the run. $K = 3$ was not used because, under the same 4-round budget, it would produce an uneven end-of-run pattern with one deep update at Round 3 followed by a final shallow-only round. The selected K values are therefore tied to the fixed round budget, not to any claim that powers of two are inherently optimal.

- $K = 1$: *Sanity check*. Every round is a full round, so the async schedule should collapse to standard synchronous FedAvg.
- $K = 2$: *Moderate cadence*. Deep parameters are refreshed every other round, giving one shallow-only round between full rounds.
- $K = 4$: *Aggressive cadence*. Deep parameters are refreshed only once within the 4-round budget, providing an upper-bound communication-saving case in this study.

The only independent variables are: (i) the system type (centralized / synchronous FL / asynchronous FL), (ii) the update cadence policy (synchronous full-model vs. periodic shallow/deep), (iii) the data distribution regime (IID vs. non-IID). All other factors (model architecture, dataset, preprocessing, training budget, aggregation rule, and random seeds) are held constant.

5 Results

5.1 System Overview and Validation

The system consists of three modules that share the same ECGCNN model (37.13M parameters) and PTB-XL data pipeline: a centralized training module, a synchronous FL module, and an asynchronous FL module. The centralized and synchronous FL modules follow established PyTorch and Flower [11] implementation patterns. The asynchronous FL module is the main technical contribution of this thesis. It implements a custom Flower server strategy with a periodic layer-wise scheduling policy, shallow and deep parameter partitioning, and per-round staleness tracking. high-level contextual overview of the three modules and their shared components is shown in Figure 5.1.

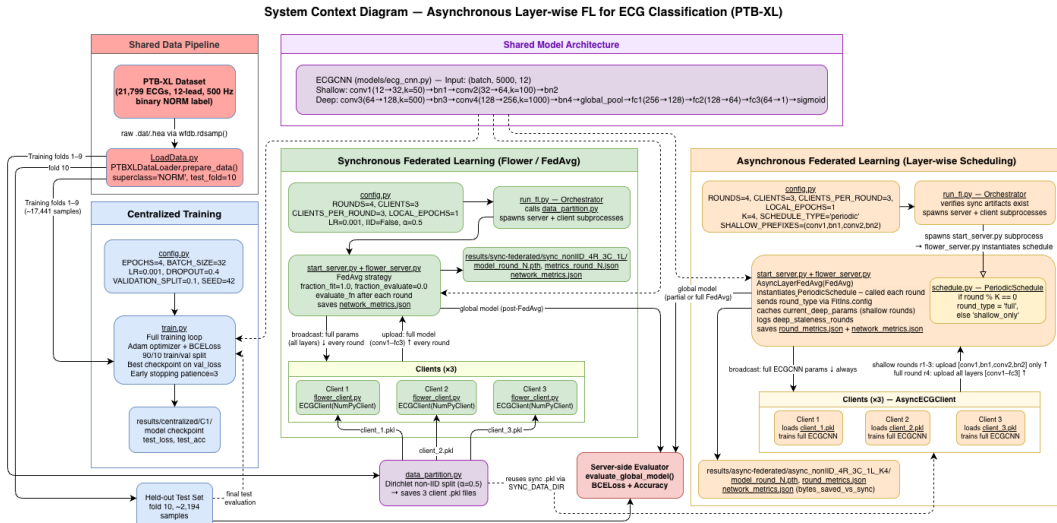


Figure 5.1: System Context Diagram showing the three modules and their shared components.

The system was validated by running the asynchronous module with $K = 1$. Setting $K = 1$ collapses the periodic schedule into standard synchronous FedAvg: all rounds are full rounds, no shallow-only rounds occur, and the expected upload reduction is 0%. If the implementation is correct, $K = 1$ async results should match synchronous FL results within the variance produced by random training dynamics. Results of this validation are reported in Section 5.4.

5.2 Centralized Baseline Results

The centralized model was trained on PTB-XL Folds 1–9 with a 90/10 training/validation split and evaluated on the held-out test set (Fold 10). Training ran for 120 minutes (4 epochs, Adam optimizer, `LEARNING_RATE=1e-3`, `BATCH_SIZE=32`) on an Apple M1 Pro using MPS acceleration. The model has 37,130,657 parameters.

- Test accuracy: 84.08%
- Test loss: 0.3267

This serves as the performance ceiling for this classification task under non-federated, non-communication-constrained conditions. Per-epoch centralized training progress is depicted in Figure 5.2.

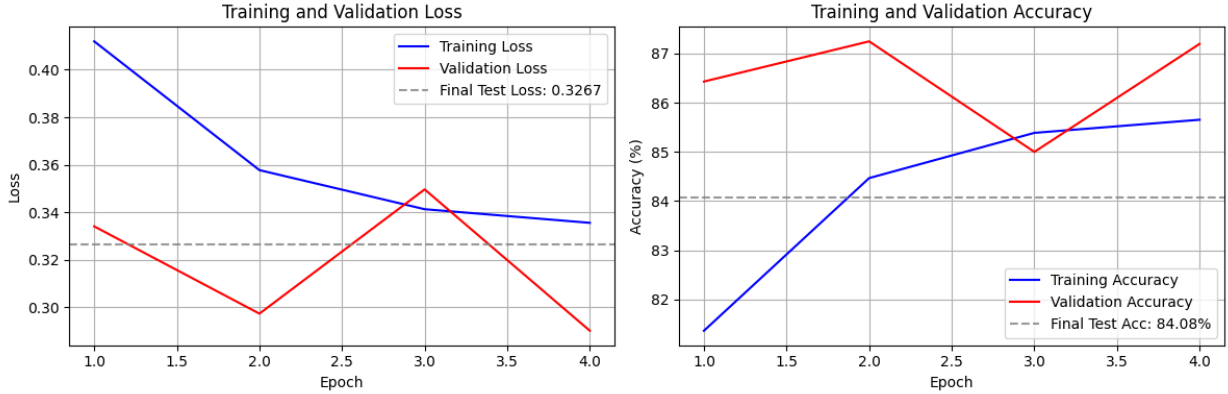


Figure 5.2: Training and validation loss and accuracy curves for the centralized ECG CNN across 4 epochs on PTB-XL (NORM vs. Abnormal).

5.3 Synchronous FL Results

Per-round accuracy and loss metrics across IID and non-IID partitioning are presented in Table 8.1 (Appendix B).

5.3.1 IID Partition

Under the IID partition, training records (Folds 1–9) were split evenly across 3 clients, with approximately 6,533 samples per client. After 4 FedAvg rounds (Round 0 is the pre-training evaluation; Rounds 1–4 are post-aggregation evaluations), the global model reached:

- Test Accuracy: 81.85%
- Test Loss: 0.4015
- Wall-clock training time: 304.4 minutes

Under standard FedAvg, the server transmits the full global model (148.5 MB) to all 3 participating clients each round, and each client returns the full updated model. Total communication over 4 rounds was 1,782,271,536 bytes upload and 1,782,271,536 bytes download, for a combined total of 3,399.41 MB exchanged over the network. Per-round training progress for the IID partition is depicted in Figure 5.3.

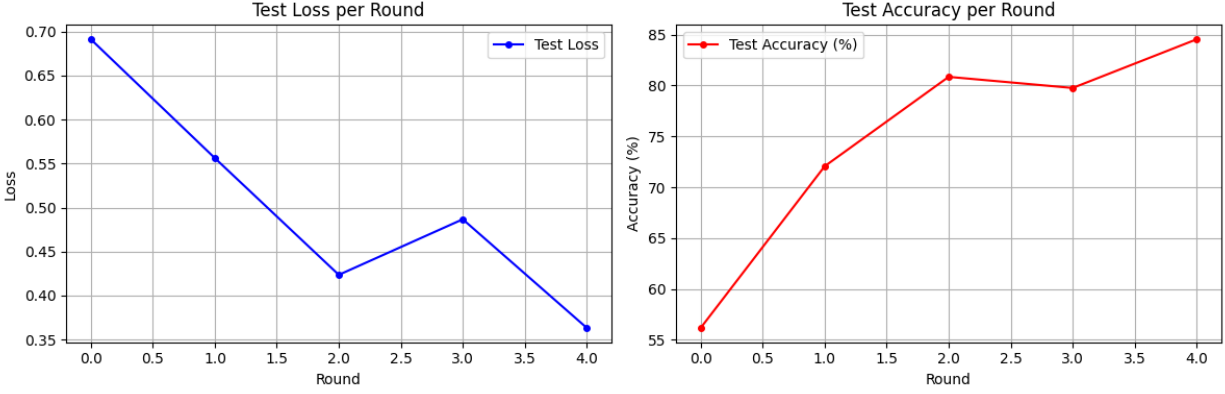


Figure 5.3: Test loss and accuracy by round for the synchronous FL model (IID partition, 3 clients, 4 rounds, 1 local epoch per round).

5.3.2 Non-IID Partition

Under the non-IID partition, data was distributed using Dirichlet allocation with $\alpha = 0.5$, producing highly skewed client datasets: Client 0 received 2,567 samples (96.5% positive class); Client 1 received 11,722 samples (24.5% positive); Client 2 received 5,312 samples (60.3% positive). The global model reached:

- Test Accuracy: 84.53%
- Test Loss: 0.3634
- Wall-clock training time: 297.8 minutes

Note: Communication cost is identical to the IID run (3,399.41 MB total) because FedAvg transmits the full model in both directions every round, regardless of data distribution.

Per-round training progress for the non-IID partition is depicted in Figure 5.4.

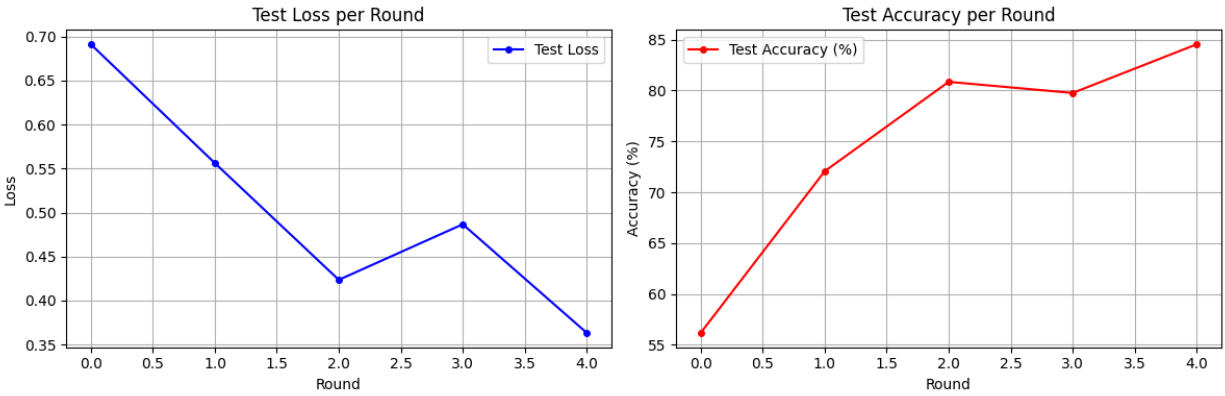


Figure 5.4: Test loss and accuracy by round for the synchronous FL model (non-IID partition, Dirichlet $\alpha = 0.5$, 3 clients, 4 rounds, 1 local epoch per round).

5.4 Asynchronous FL Results

5.4.1 Implementation Validation ($K = 1$, IID)

With $K = 1$, the periodic schedule assigns every round as a full round: all 4 rounds transmit both shallow and deep parameters. This makes the async schedule equivalent to standard synchronous FedAvg and serves as an integration test for the implementation. It is a sanity check for code-path equivalence, not a convergence validation.

The $K = 1$ run (Figure 5.5) produced 83.71% test accuracy with a test loss of 0.3662, completing in 277.5 minutes. Upload reduction versus the synchronous baseline was 0.00%, with total communication at 3,399.50 MB. The synchronous IID baseline achieved 81.85% accuracy. The difference of 1.86 percentage points falls within the expected variance from random client sampling and local training dynamics. Communication cost matched the synchronous baseline to within 0.003%.

The round-by-round accuracy trajectory of Async $K = 1$ tracks the synchronous IID trajectory closely across all 5 rounds, confirming that the asynchronous module produces equivalent results when the schedule returns to full-model updates.

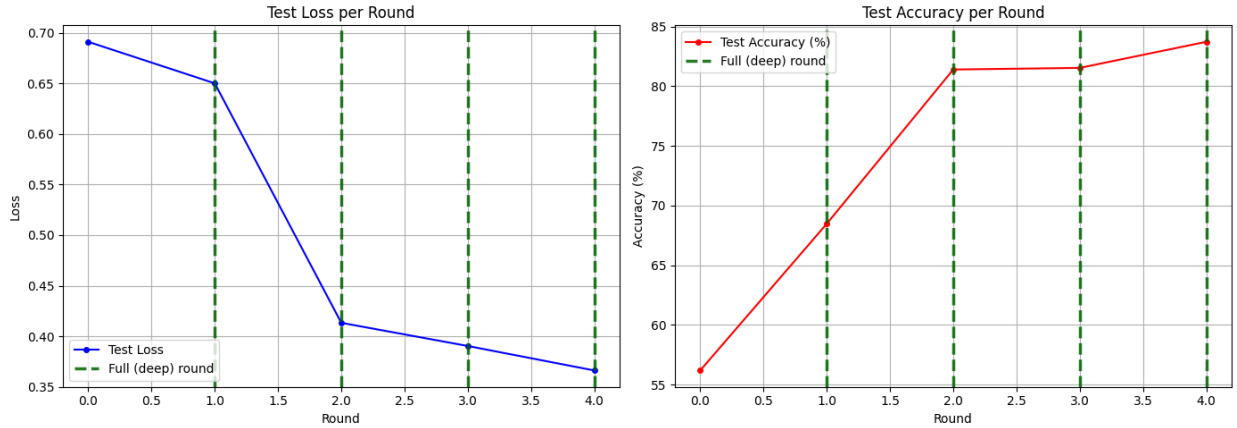


Figure 5.5: Test loss and accuracy by round for the async FL model (IID, $K = 1$, validation run). Round types: all full.

5.4.2 IID Experiments ($K = 2$ and $K = 4$)

$K = 2$ (IID). With $K = 2$, the periodic schedule alternates between shallow-only and full rounds: Round 1 (shallow), Round 2 (full), Round 3 (shallow), Round 4 (full). In this single-seed run, the model reached 83.08% test accuracy with a test loss of 0.3675 in 289.4 minutes (Figure 5.6).

$K = 4$ (IID). With $K = 4$, three consecutive shallow-only rounds precede the single full round: Rounds 1–3 (shallow), Round 4 (full). In this single-seed run, the model reached 79.21% test accuracy with a test loss of 0.4285 in 298.8 minutes (Figure 5.7).

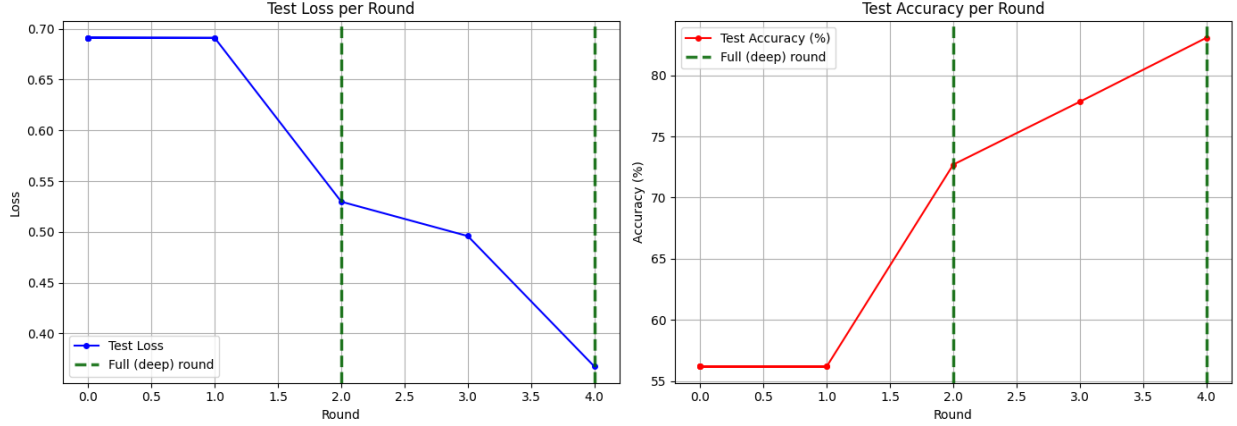


Figure 5.6: Test loss and accuracy by round for the async FL model (IID, $K = 2$). Round types: shallow, full, shallow, full.

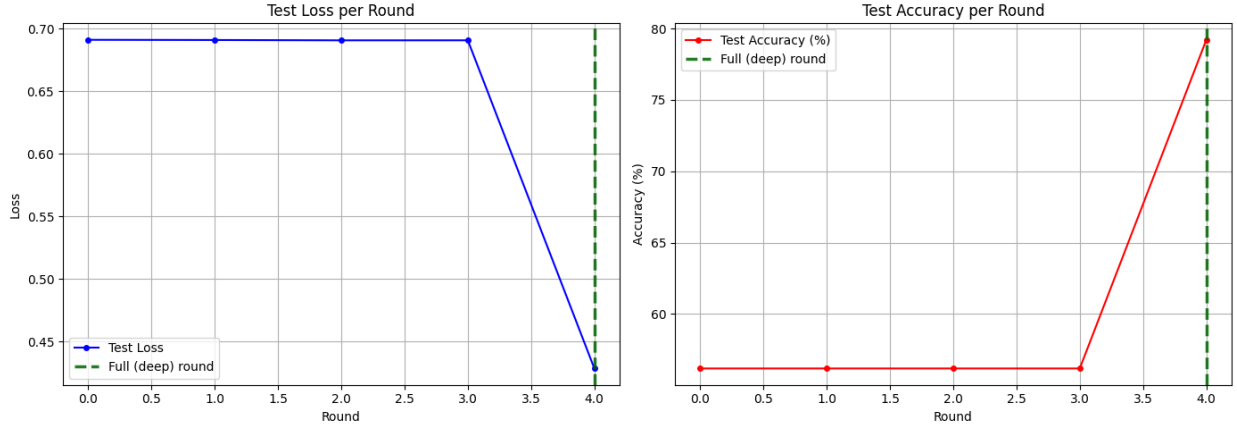


Figure 5.7: Test loss and accuracy by round for the async FL model (IID, $K = 4$). Round types: shallow, shallow, shallow, full.

5.4.3 Non-IID Experiments ($K = 2$ and $K = 4$)

Both non-IID async experiments used the same Dirichlet partition ($\alpha = 0.5$) as the synchronous non-IID baseline: Client 0 (2,567 samples), Client 1 (11,722 samples), Client 2 (5,312 samples).

$K = 2$ (**non-IID**). The round pattern is identical to the IID $K = 2$ case (shallow, full, shallow, full). In this single-seed run, the model reached 84.62% test accuracy with a test loss of 0.3728 in 326.8 minutes (Figure 5.8).

$K = 4$ (**non-IID**). The round pattern is identical to the IID $K = 4$ case (shallow, shallow, shallow, full). In this single-seed run, the model reached 83.89% test accuracy with a test loss of 0.4004 in 295.8 minutes (Figure 5.9).

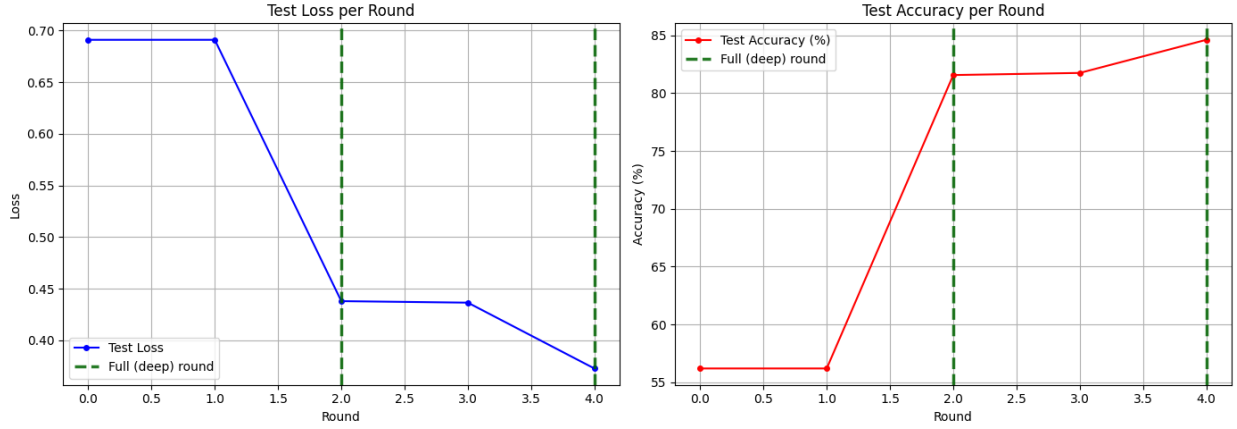


Figure 5.8: Test loss and accuracy by round for the async FL model (non-IID, $K = 2$). Round types: shallow, full, shallow, full.

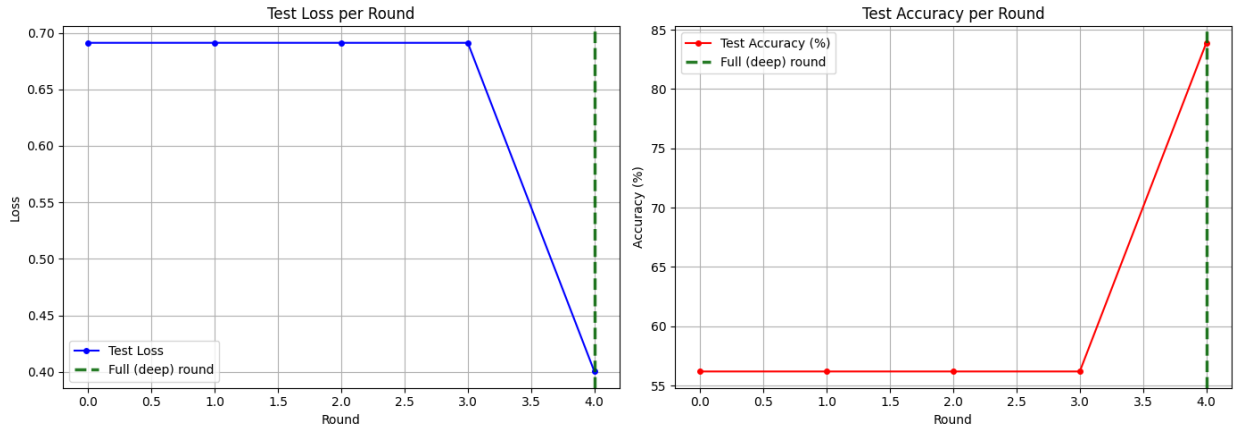


Figure 5.9: Test loss and accuracy by round for the async FL model (non-IID, $K = 4$). Round types: shallow, shallow, shallow, full.

5.4.4 Shallow-Round Stagnation

Tables 8.2 and 8.3 show the same pattern across all async experiments: test accuracy does not meaningfully change during shallow-only rounds. In every configuration tested (IID and non-IID, $K = 2$ and $K = 4$), accuracy remains at 56.19% throughout those rounds, which is the same value recorded at Round 0 before any client training has occurred. Accuracy changes appear only after rounds that include deep-parameter aggregation.

The parameter split helps explain why this happens. The shallow partition is 897,936 bytes, or 0.60% of the full model state dict (148,526,500 bytes), while the deep partition is 147,628,564 bytes (99.40%). Under $K = 2$, deep parameters are at most 1 round stale because each shallow-only round is immediately followed by a full round. Under $K = 4$, deep parameters remain stale for 3 consecutive rounds before the first full update at Round 4.

The pattern is the same in both IID and non-IID settings, which suggests that the main driver is not the data split but the partition design itself. With such an extreme asymmetry, shallow-only rounds are updating only a tiny fraction of the trainable state while the 99.40% deep component stays fixed. In that sense, the stagnation is more naturally read as a consequence of the chosen shallow/deep boundary than as a separate empirical finding.

This matters for how the schedule should be designed. K should be chosen together with the parameter split, and the shallow/deep boundary should not be defined by layer count alone. If the shallow group is too small, increasing the frequency of shallow-only rounds mainly saves communication by skipping updates to the part of the model that still carries most of the learning capacity.

An Information Bottleneck (IB) interpretation is still possible. The shallow layers may reach a sufficient representation for this ECG task very early, so further shallow-only aggregation adds little. However, that remains a secondary interpretation rather than a formal IB result. Distinguishing between signal simplicity and partition asymmetry would require experiments with larger shallow partitions and non-ECG signal types.

5.5 Communication Cost Comparison

The full model state dict is 148.5 MB. The shallow partition (`conv1`, `bn1`, `conv2`, `bn2`) is 0.9 MB (0.60% of the full model). The deep partition (all remaining layers: `conv3`, `bn3`, `conv4`, `bn4`, `fc1`, `fc2`, `fc3`) is 147.6 MB (99.40%).

Download cost is unchanged across all async configurations because the server broadcasts the full model to participating clients each round, regardless of the round type. All upload savings come from omitting deep parameter transmission on shallow-only rounds.

At $K = 1$, total communication matches the synchronous baseline (3,399.50 MB vs. 3,399.41 MB) and upload reduction is 0%, as expected for the sync-equivalent sanity check. At $K = 2$, total communication falls to 2,554.76 MB, a 24.8% reduction, while upload is reduced by 49.70%. Final accuracy in the single-seed runs is 83.08% under IID and 84.62% under non-IID. At $K = 4$, total communication falls to 2,132.40 MB and upload is reduced by 74.55%, with final accuracy of 79.21% under IID and 83.89% under non-IID.

Within this 3-client, 4-round simulation, these results show that periodic layer-wise scheduling can reduce client upload cost substantially, but the accompanying accuracy values should be interpreted cautiously because they come from a minimal experimental budget and a single seed.

5.6 Novelty of Results

Comparison with Hartmann (2018). Hartmann [5] demonstrated that communication overhead in FL can be reduced by applying uniform compression across all model parameters: sparse masks (randomly transmitting a fraction p of parameters), probabilistic quantization (encoding updates in fewer bits), and structured low-rank updates. Each of these techniques applies the same transformation to every parameter regardless of its layer depth. The system described in this thesis takes a different approach: it varies the transmission frequency of parameter groups by depth rather than compressing parameter values. Shallow parameters (`conv1`, `bn1`, `conv2`, `bn2`) are transmitted every round; deep parameters are transmitted every K rounds. The two approaches are not mutually exclusive; layer-wise scheduling could, in principle, be combined with Hartmann’s compression methods for additional savings.

Comparison with Yuan et al. (2020). Yuan et al. [7] reported a much larger communication reduction than this study: 99.8% per-iteration traffic reduction relative to FedAvg (from 1.36 GB to 3.2 MB for 16 devices) on a PhysioNet arrhythmia task. By comparison, the best result in this thesis at $K = 2$ is a 49.70% upload reduction, which corresponds to a 24.8% reduction in total communication because downlink remains unchanged. The magnitude of improvement here is therefore more modest. The mechanisms also differ: Yuan et al. partition computation spatially, with a shallow sub-network on the device and a deeper network on the server, whereas this thesis keeps the full model on the client and reduces communication by lowering the upload frequency of deep parameters. The numbers are therefore not directly comparable, but the comparison helps place the present contribution in context.

Contribution of this study. The main contribution of this work is not the absolute size of the communication reduction, but the demonstration of a simple periodic, layer-wise scheduling mechanism under a controlled ECG classification setup. In this limited setting, the mechanism reduces total communication by 24.8% at $K = 2$ while halving upload volume, and pushing the cadence further to $K = 4$ increases savings but weakens learning. The results also show that when the shallow partition is only 0.60% of the model, shallow-only rounds may contribute little measurable learning. That observation is best treated as a design consequence of the chosen partition, with implications for how future work should select K and define the shallow/deep boundary.

6 Discussion

6.1 Threats to Validity

6.1.1 Dataset Preprocessing. PTB-XL was preprocessed by its creators [9] through spike removal, resampling ($400 \rightarrow 500$ Hz), and format conversion. This may have stripped high-frequency content, widening the gap between these clean clinical signals and the noisier outputs typical of real IoT wearables (motion artifacts, lower SNR, inconsistent electrode contact). Relative comparisons between sync and async architectures remain valid since all experiments share the same pipeline. However, the shallow-only stagnation observed in Section 5.4 was found on preprocessed data; noisier signals could shift the convergence behaviour of shallow versus deep layers, potentially altering the viable deep-update cadence (K).

6.1.2 Hardware and Simulation Environment. All experiments ran on a 2021 MacBook M1 Pro with clients simulated as separate processes. Real network latency, device heterogeneity, packet loss, and wireless bandwidth constraints were not captured; communication cost is measured in bytes, not wall-clock time. The reported upload savings (49.70% at $K = 2$, 74.55% at $K = 4$) translate to proportional time savings only if bandwidth is the bottleneck. This is plausible for IoT devices [5], but not confirmed here. In practice, radio startup and association overhead often dominates energy cost, so byte savings alone may not accurately reflect device-level energy impact.

6.1.3 Data Volume and Federation Size. PTB-XL’s 21,799 records were split across 3 simulated clients ($\sim 6,500$ samples each under IID) and trained for only 4 rounds. The setup is sufficient for isolating the communication effect of the scheduling rule, but it is too small to support broad claims about convergence behaviour or expected accuracy in larger federations. Those quantities may change with more clients, smaller local datasets, repeated client participation, or longer training horizons.

6.2 Implications of Research Results

6.2.1 Impact on Related Research. These results support layer-wise update scheduling as a distinct axis for reducing FL communication cost, separate from the compression approaches in Hartmann [5] and the computation-partitioning approach of Yuan et al. [7]. In this study, the practical gain at $K = 2$ is a 24.8% reduction in total communication, alongside a 49.70% reduction in upload volume, without modifying parameter values. This is a more modest reduction than the 99.8% per-iteration traffic reduction reported by Yuan et al. [7], but it is achieved within a standard FedAvg-style client/server setup rather than a split-computation system. The shallow-round stagnation discussed in Section 5.4 should be interpreted cautiously: in this thesis it is more plausibly a consequence of the extreme partition asymmetry (0.60% shallow) than a broadly general property of layer-wise scheduling.

6.2.2 Impact on Practice. In settings resembling this minimal experiment, $K = 2$ can halve client upload volume while staying within 1 percentage point of the synchronous baseline. The benefit is asymmetric: download cost is unchanged since the server always broadcasts the full model. Deployments where uplink is the tighter constraint (common in IoT, where uplink is typically slower than downlink [5]) would be the most natural candidates for this kind of schedule.

6.3 Limitations of Results

Fixed K values. Only $K \in \{1, 2, 4\}$ were evaluated. The optimal K for a given deployment depends on model architecture, dataset, client count, and training budget. An adaptive schedule (e.g., increasing K when loss plateaus) could yield better trade-offs but was out of scope.

Convergence guarantees. The decoupled schedule alters the server update, not just communication cost. As shown in Appendix C.7, shallow-only rounds update $(\theta_S^{t+1}, \hat{\theta}_D^t)$ instead of $(\text{Agg}_S^t, \text{Agg}_D^t)$, i.e., a partial FedAvg step on shallow parameters with deep parameters frozen. This restricts optimization to $\mathcal{L}(\theta_S, \hat{\theta}_D^t)$, preventing coordinated updates across S and D until a full round. The result is a jagged path: shallow updates adapt to stale deep features, deep client updates are discarded, and mismatch accumulates. Thus, the strong $K = 2$ result does not guarantee convergence; over longer runs or higher heterogeneity, it may cause slower convergence, drift, or poorer minima.

Random seed sensitivity. All results use a fixed seed (RANDOM_SEED=42). No multi-seed analysis was completed within the available compute budget, so the $K = 2$ accuracy values may reflect favourable run-to-run variation. A follow-up study should repeat each condition across multiple seeds and report mean and variance.

Single model architecture. All experiments used ECGCNN. Shallow: conv1, bn1, conv2, bn2 (0.60% of parameters). Deep: remaining layers (99.40%). This extreme asymmetry is architecture-specific. Models with more balanced layer distributions, or Transformers where early layers carry global contextual information, could behave differently during shallow-only rounds.

Limited heterogeneity regime. Non-IID experiments used Dirichlet $\alpha = 0.5$ (moderate label skew). More extreme heterogeneity (e.g., $\alpha = 0.1$) or imbalanced dataset sizes were not tested. Under stronger skew, stale deep parameters compounding with biased local updates could cause instability not visible at $\alpha = 0.5$.

Single dataset. Only PTB-XL was used. Communication savings are architecture-dependent and generalize across datasets, but accuracy trade-offs may not.

Upload-only savings. All reported savings cover the upload (client→server) path only. At $K = 2$, total communication drops from 3,399 MB to 2,555 MB, a 24.8% reduction, not 49.70%.

6.4 Generalisability of Results

Communication savings generalize to any FL setup using the same model, as they depend on parameter split and K , not data. Accuracy trade-offs are less general and depend on dataset, architecture, training duration, and federation size. While $K = 2$ remains within 1% accuracy under both IID and non-IID settings, this may not hold under longer or larger-scale deployments.

The observed shallow-round stagnation is tied to the extreme parameter split and ECG domain and should not be assumed elsewhere. Results apply most directly to horizontal FL with a small number of health IoT clients performing binary ECG classification. Extending to other FL types (vertical, hierarchical), biosignals (PPG, EEG), or domains requires further validation.

6.5 Value of the Controlled Small-Scale Setup

The constrained experiment design (3 clients, 4 rounds, fixed seeds, shared hardware) provides a practical advantage for this stage of research. Because experimental variance is intentionally reduced, communication differences can be attributed more directly to the schedule definition itself (periodic deep synchronization with shallow updates every round). This supports transparent debugging, reproducible accounting of byte-level costs, and clear sanity checks such as the $K = 1$ sync-equivalence run.

For a proof-of-concept thesis, this controlled setting is appropriate for answering a bounded question: whether the proposed scheduling mechanism can produce measurable communication savings without immediate utility collapse in a baseline ECG FL setup. The study therefore does not claim field-wide performance guarantees; instead, it contributes a validated mechanism, an interpretable benchmark protocol, and a set of falsifiable hypotheses for larger, less controlled future evaluations.

7 Conclusions

Standard synchronous Federated Learning (FL) requires every client to upload the full model each round, creating communication overhead that is especially burdensome for IoT health devices with bandwidth and compute constraints [3], [6]. This thesis investigated whether an asynchronous, layer-wise weight-updating schedule could reduce that overhead while preserving acceptable model utility on an ECG classification task. The objectives were to: establish a centralized performance baseline on PTB-XL (O1), implement a synchronous FL baseline with FedAvg (O2), design and implement a periodic asynchronous schedule that transmits shallow and deep parameters at different frequencies (O3), and evaluate the communication-accuracy trade-off under both IID and non-IID data distributions (O4).

All four planned objectives were completed within the scoped design. The centralized baseline achieved 84.08% accuracy (Section 5.2). Synchronous FL achieved 81.85% (IID) and 84.53% (non-IID) with 3,399 MB total communication over 4 rounds (Table 8.1). At $K = 2$, the async schedule reduced total communication to 2,554.76 MB, a 24.8% reduction, while reducing client upload volume by 49.70% and keeping accuracy within 1 percentage point of the synchronous baselines: 83.08% (IID) and 84.62% (non-IID) (Table 8.4). At $K = 4$, upload reduction reached 74.55% with accuracy of 79.21% (IID) and 83.89% (non-IID).

Across all async experiments, accuracy remained unchanged during shallow-only rounds (Tables 8.2–8.3). The shallow partition is 0.60% of model parameters; updating it alone produced no measurable learning. Accuracy improved only after rounds that included deep parameter aggregation (Section 5.4).

Within the tested non-IID regime ($\alpha = 0.5$), the $K = 2$ schedule stayed close to the synchronous baseline while cutting upload cost by about half. More broadly, periodic layer-wise scheduling reduced communication in this controlled setup by changing transmission frequency rather than parameter values or computation placement, which makes it complementary to the approaches of Hartmann [5] and Yuan et al. [7].

The primary value of this thesis is therefore not to prove superiority of asynchronous FL in general, but to establish a clear and reproducible proof-of-concept showing how schedule design alone can shape communication-utility trade-offs under constrained experimental conditions.

8 Future Work and Lessons Learnt

8.1 Future Work

Adaptive periodic scheduling. A natural extension is to keep the periodic schedule, but allow K to change in response to simple server-side signals. One concrete rule would start from $K = 2$ and evaluate adjustment only after full rounds. Let L_{full} be the drop in validation loss after a full round, and let s_t be the current deep staleness. If L_{full} remains below a small threshold for m consecutive full rounds and s_t stays below a preset limit, increase K by 1 up to K_{max} to save communication. If validation loss worsens after a shallow-only round, or if the next full round produces a large corrective jump, decrease K by 1 down to K_{min} to refresh deep parameters sooner. This would preserve the same scheduling framework used here while making the cadence responsive to evidence of under-updating or over-updating.

Partition-size ablation. The most important follow-up to Section 5.4 is to vary the shallow/deep boundary systematically while keeping the model family and training budget fixed. Rather than selecting shallow layers by position alone, future work should define target shallow shares such as 5%, 10%, 20%, and 40% of parameters, then compare $K = 2$ and $K = 4$ under each partition. This would directly test whether the stagnation observed here is caused mainly by the extreme 0.60% shallow share or by the ECG task itself. It would also produce a more useful design rule for practice: choose K jointly with the parameter share assigned to the high-frequency update group.

Hardware-grounded evaluation. The communication metric in this thesis is bytes exchanged under simulation. A final step toward deployment would be to run the same schedule on actual IoT hardware or a network emulator that exposes latency, packet loss, and radio wake-up cost. That would allow K to be selected against a joint objective such as:

$$J = \alpha \cdot \text{bytes} + \beta \cdot \text{transmissions},$$

which better reflects real wireless energy use than payload size alone.

8.2 Lessons Learnt

Partition imbalance limits shallow updates. When the shallow partition contains $<1\%$ of parameters, shallow-only updates produce no measurable accuracy change (Section 5.4), across both IID and non-IID settings. Partitioning should be based on parameter share, not layer count, and must allocate a meaningful fraction to the shallow group.

Scheduling interacts with data heterogeneity. Under the tested non-IID setting ($\alpha = 0.5$), async $K = 2$ reached 84.62% accuracy, slightly above the 84.53% synchronous baseline, while $K = 4$ non-IID (83.89%) exceeded $K = 4$ IID (79.21%) (Section 5.5, Table 8.4). This shows that the interaction between schedule and data heterogeneity is not straightforward in this small experiment, but it is not enough to claim general robustness to non-IID data.

Upload and total communication differ. At $K = 2$, upload volume is reduced by 49.70%, but total communication decreases by only 24.8% because the server continues to broadcast the full model each round. Reporting upload reduction alone would overstate practical savings. Both upload and total communication should be reported explicitly in future work.

Acknowledgements

The author thanks Prof. Zubair Fadlullah, Department of Computer Science, Western University, for supervision and guidance throughout this research project.

Appendix

Appendix A: Tables

Table 8.1: Round-by-round test accuracy and loss for both synchronous FL experiments.

Round	Sync IID Acc.	Sync IID Loss	Sync non-IID Acc.	Sync non-IID Loss
0	56.19%	0.6911	56.19%	0.6911
1	68.47%	0.6501	72.07%	0.5562
2	81.39%	0.4150	80.85%	0.4236
3	80.44%	0.4312	79.75%	0.4867
4	81.85%	0.4015	84.53%	0.3634

Table 8.2: Round-by-round test accuracy for IID experiments. Asterisks (*) mark shallow-only rounds.

Round	Sync IID	Async $K = 1$	Async $K = 2$	Async $K = 4$
0	56.19%	56.19%	56.19%	56.19%
1	68.47%	68.47%	56.19%*	56.19%*
2	81.39%	81.39%	72.70%	56.19%*
3	80.44%	81.53%	77.84%*	56.19%*
4	81.85%	83.71%	83.08%	79.21%

Table 8.3: Round-by-round test accuracy for non-IID experiments. Asterisks (*) mark shallow-only rounds.

Round	Sync non-IID	Async $K = 2$	Async $K = 4$
0	56.19%	56.19%	56.19%
1	72.07%	56.19%*	56.19%*
2	80.85%	81.57%	56.19%*
3	79.75%	81.76%*	56.19%*
4	84.53%	84.62%	83.89%

Table 8.4: Communication cost and model utility across all federated experiments.

Experiment	Total (MB)	Upload (bytes)	Download (bytes)	Upload Red.	Final Acc.	Final Loss
Sync IID	3,399.41	1,782,271,536	1,782,271,536	– (baseline)	81.85%	0.4015
Sync non-IID	3,399.41	1,782,271,536	1,782,271,536	– (baseline)	84.53%	0.3634
Async $K = 1$ IID	3,399.50	1,782,318,000	1,782,318,000	0.00%	83.71%	0.3662
Async $K = 2$ IID	2,554.76	896,546,616	1,782,318,000	49.70%	83.08%	0.3675
Async $K = 4$ IID	2,132.40	453,660,924	1,782,318,000	74.55%	79.21%	0.4285
Async $K = 2$ non-IID	2,554.76	896,546,616	1,782,318,000	49.70%	84.62%	0.3728
Async $K = 4$ non-IID	2,132.40	453,660,924	1,782,318,000	74.55%	83.89%	0.4004

Appendix B: Detailed Methodology

B.1 Controlled Variables

Controlled variables ensure that centralized, synchronous, and asynchronous training are compared fairly so that any observed differences can be attributed to the network communication scheduling strategy (NCSS) rather than confounding factors.

- **CNN Architecture:** NUM_LEADS=12, DROPOUT_RATE=0.4, MODEL_IMPL=models/ecg_cnn.py
- **Dataset and Preprocessing:** SUPERCLASS=NORM, TEST_FOLD=10, NORMALIZE=True, VALIDATION_SPLIT=0.1
- **Training Budget and Core Hyperparameters:** NUM_EPOCHS=4 (centralized), NUM_ROUNDS=4 (federated), LOCAL_EPOCHS=1 (federated), BATCH_SIZE=32, LEARNING_RATE=1e-3
- **Random Seed and Reproducibility:** RANDOM_SEED=42, PARTICIPATION_SEED=42
- **Aggregation Semantics (FedAvg Baseline):** AGGREGATION_RULE=FedAvg, AGGREGATION_SCOPE=full_model

B.2 Independent Variables

Independent variables are experimentally manipulated to probe how asynchronous, layer-wise update scheduling affects communication overhead and model utility.

- **System Type:** SYSTEM_TYPE $\in$ {centralized, synchronous_fl, asynchronous_fl}
- **Update Cadence Policy (Primary Factor):** SCHEDULE_TYPE=periodic, DEEP_EVERY_N_ROUNDS (K) = 2 $\Rightarrow$ Shallow:Deep Ratio = 1 : 2
- **Layer Partitioning (Shallow vs. Deep):** SHALLOW_PREFIXES = {conv1., bn1., conv2., bn2.}, DEEP_PARAMS = complement(SHALLOW_PREFIXES)
- **Data Distribution Regime:** IID $\in$ {True, False}, DIRICHLET_ALPHA=0.5 (when IID=False)
- **Client Participation Regime:** NUM_CLIENTS=3, CLIENTS_PER_ROUND=1, PARTICIPATION_SEED=42
- **Bandwidth Regime:** SIMULATED_BANDWIDTH_BPS=B (finite for constrained bandwidth, None for unlimited)

B.3 Dependent Variables

Dependent variables are measured outcomes used to evaluate communication efficiency, model utility, and synchronization behaviour under each configuration.

- **Communication Cost Metrics (Primary Outcomes):** TOTAL_COMMUNICATION_BYTES, TOTAL_COMMUNICATION_MB, BYTES_UP_PER_ROUND, BYTES_DOWN_PER_ROUND, BYTES_PER_CLIENT, UPLOAD_REDUCTION_PCT, DEEP_STALE_ROUNDS
- **Model Utility and Convergence Metrics (Secondary Outcomes):** FINAL_TEST_LOSS, FINAL_TEST_ACCURACY, ACCURACY_CURVE, LOSS_CURVE, AUROC, F1_SCORE
- **Runtime and Synchronization Overhead Proxies (Tertiary Outcomes):** TOTAL_TRAINING_TIME, ROUND_TIME_STATS, ROUNDS_TO_TARGET_METRIC, TIME_TO_TARGET_METRIC

B.4 Centralized Module Setup

To establish a robust baseline, the PTB-XL dataset was partitioned into three distinct subsets. Folds 1–9 are aggregated to form the primary training pool, while Fold 10 is reserved as the held-out test set. Within the training loop, a 90/10 split was applied to the training pool: 90% for gradient updates and 10% as a validation set to monitor generalization error. The validation set enables early stopping, which terminates training when validation loss stops decreasing, ensuring the baseline model represents the best generalization performance rather than memorization.

B.5 Synchronous FL Module Setup

The synchronous FL experiments were simulated on a single machine, where each client runs as a distinct process orchestrated to run concurrently. The system is implemented in Python using PyTorch and Flower (`flwr`).

IID Partitioning Rationale. IID client partitions are included as a controlled, diagnostic condition rather than a realistic deployment assumption. By removing data heterogeneity, IID experiments isolate algorithmic behaviour, allowing differences between centralized, synchronous FL, and asynchronous FL to be attributed to aggregation, communication, and update scheduling mechanisms rather than skewed client data. Under IID partitioning, federated learning should approximate centralized training when the model, hyperparameters, and effective training budget are matched. Large discrepancies in this regime would indicate implementation or optimization issues rather than inherent limitations of the method. IID experiments therefore serve as a sanity check and stability baseline against which the impact of heterogeneous data and asynchronous scheduling can be properly evaluated.

Data Distribution. The training data (Folds 1–9) was distributed across clients using an IID partitioning strategy. The aggregated training records were shuffled and split evenly, assigning approximately 3,920 samples per client. Unlike the centralized approach, clients do not reserve a local validation split as 100% of their partition is used for local training.

Server and Client Roles. The central server maintains the global model, selects clients, coordinates communication rounds, and aggregates client model updates using FedAvg. Clients perform local training and communicate only model parameters (weights and biases) to the server. At the start of each round, the server distributes the current global model to participating clients. Each client trains locally for 1 epoch using the Adam optimizer with `BATCH_SIZE=32`, then transmits updated parameters back. The server produces the next global model by computing a sample-size-weighted average of received client parameters.

Training Protocol. Training used one global model and three local client models over four global rounds. Round 0 evaluates the freshly initialized ECGCNN before any client training. Rounds 1–4 produce evaluations after each FedAvg aggregation.

B.6 Asynchronous FL Module Setup

Table 8.5: Notation for the asynchronous periodic deep-layer schedule and its correspondence to the implementation.

Symbol	Description and implementation
T	Total number of communication rounds – <code>NUM_ROUNDS</code>
K	Deep-layer synchronization period – <code>DEEP_EVERY_N_ROUNDS</code>
θ	Full global model parameters (server state) – <code>model.state_dict()</code>
θ_S	Shallow parameter partition – keys matching <code>SHALLOW_PREFIXES</code> , indices in <code>shallow_idx</code> s
θ_D	Deep parameter partition; complement of <code>shallow_idx</code> s, stored via <code>deep_idx</code> s
$\hat{\theta}_D^t$	Server-side deep cache at round t – <code>current_deep_params</code>
$\mathcal{P}_t$	Set of participating clients in round t – sampled in <code>configure_fit</code>
w_i^t	Aggregation weight for client i in round t (proportional to local examples) – <code>num_examples / total_examples</code> in <code>_weighted_average_arrays</code>
$\theta_{S,i}^t$	Client i ’s shallow parameters after local training in round t – subset of <code>get_parameters()</code>
$\theta_{D,i}^t$	Client i ’s deep parameters after local training in round t – subset of <code>get_parameters()</code> corresponding to <code>deep_idx</code> s
Agg_S^t	FedAvg aggregate of shallow parameters at round t – <code>_weighted_average_arrays</code> on shallow arrays
Agg_D^t	FedAvg aggregate of deep parameters at round t (full rounds only) – analogous on deep arrays
$\mathbb{I}[t \bmod K = 0]$	Indicator of a full (deep) round under <code>PeriodicSchedule</code> – <code>if server_round % DEEP_EVERY_N_ROUNDS == 0</code>
$ \theta_S $	Byte size of shallow partition; measured once as <code>_shallow_bytes</code> from model <code>state_dict()</code>
$ \theta_D $	Byte size of deep partition; measured once as <code>_deep_bytes</code>
$C_{\text{up}}(t)$	Total uplink bytes at round t – <code>bytes_up_total</code> in <code>_log_round</code>
$C_{\text{down}}(t)$	Total downlink bytes at round t – <code>bytes_down_total</code> in <code>_log_round</code>

Intuition. Shallow layers are sent more often because they form the foundation of the neural network. They learn basic features that deep layers build upon. When shallow layers improve, the entire network benefits. Deep layers, which learn slower-changing, task-specific patterns, do not need to be synchronized as frequently.

Server-Side Evaluation. After each communication round, the aggregated global model is evaluated only on the server using the fixed held-out test set (same split as centralized and synchronous baselines). The server loads updated global parameters, runs inference on the test set, and logs loss and accuracy per round. No client-side evaluation is performed (`fraction_evaluate=0.0`, `min_evaluate_clients=0`). This design ensures consistency (all methods evaluated on the same test set), adds no communication cost (evaluation does not involve client-server exchanges), and enables fair comparison (performance differences reflect training and aggregation strategy, not evaluation protocol).

B.7 Formalized Asynchronous Schedule

Let $T \in \mathbb{N}$ denote the total number of communication rounds and let $K \in \mathbb{N}$ denote the deep-layer synchronization period. Let the full model parameters be partitioned as

$$\boldsymbol{\theta} = (\boldsymbol{\theta}_S, \boldsymbol{\theta}_D), \quad (8.1)$$

where $\boldsymbol{\theta}_S$ is the *shallow* parameter tensor (layers matching `SHALLOW_PREFIXES`) and $\boldsymbol{\theta}_D$ is the *deep* parameter tensor (all remaining layers).

At round $t \in \{1, \dots, T\}$, let $\mathcal{P}_t$ be the set of participating clients, and let $\boldsymbol{\theta}_{S,i}^t, \boldsymbol{\theta}_{D,i}^t$ denote client i 's shallow and deep parameters *after* local training in that round. The server performs a FedAvg-style aggregation:

$$\text{Agg}_S^t = \sum_{i \in \mathcal{P}_t} w_i^t \boldsymbol{\theta}_{S,i}^t, \quad \text{Agg}_D^t = \sum_{i \in \mathcal{P}_t} w_i^t \boldsymbol{\theta}_{D,i}^t, \quad \sum_{i \in \mathcal{P}_t} w_i^t = 1. \quad (8.2)$$

We define the indicator function $\mathbb{I}[\cdot]$ and let $\hat{\boldsymbol{\theta}}_D^t$ be the server-side deep cache (the most recently aggregated deep parameters). For the `PeriodicSchedule`, the server update at round t is:

$$\boldsymbol{\theta}_S^{t+1} = \text{Agg}_S^t, \quad (8.3)$$

$$\boldsymbol{\theta}_D^{t+1} = \mathbb{I}[t \bmod K = 0] \text{Agg}_D^t + (1 - \mathbb{I}[t \bmod K = 0]) \hat{\boldsymbol{\theta}}_D^t, \quad (8.4)$$

$$\hat{\boldsymbol{\theta}}_D^{t+1} = \mathbb{I}[t \bmod K = 0] \boldsymbol{\theta}_D^{t+1} + (1 - \mathbb{I}[t \bmod K = 0]) \hat{\boldsymbol{\theta}}_D^t. \quad (8.5)$$

In simpler terms, if $t \bmod K = 0$ (a *full* round), the server aggregates *both* shallow S and deep D partitions and refreshes the deep cache $\hat{\boldsymbol{\theta}}_D$. Otherwise (a *shallow-only* round), the server aggregates only the shallow partition and reuses the cached deep parameters. The server aggregates shallow partitions S in every round t .

Communication Cost Model. Let $|\boldsymbol{\theta}_S|$ and $|\boldsymbol{\theta}_D|$ denote the byte sizes of the shallow and deep parameter partitions. The uplink (client→server) cost at round t is:

$$C_{\text{up}}(t) = |\mathcal{P}_t| \left(|\boldsymbol{\theta}_S| + |\boldsymbol{\theta}_D| \mathbb{I}[t \bmod K = 0] \right) \quad [\text{bytes}], \quad (8.6)$$

and the downlink (server→client) cost (broadcasting the full model each round) is:

$$C_{\text{down}}(t) = |\mathcal{P}_t| \left(|\boldsymbol{\theta}_S| + |\boldsymbol{\theta}_D| \right) \quad [\text{bytes}]. \quad (8.7)$$

B.8 Source Code

The full source code for this thesis is available on GitHub at [LlamzonAmazon/asynchronous-fl](https://github.com/LlamzonAmazon/asynchronous-fl) [12]. This is the open-source codebase developed to evaluate asynchronous weight-updating federated learning in this research.

References

- [1] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agüera y Arcas, “Communication-efficient learning of deep networks from decentralized data,” in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*, ser. Proceedings of Machine Learning Research, vol. 54, 2017, pp. 1273–1282. [Online]. Available: <https://proceedings.mlr.press/v54/mcmahan17a.html>
- [2] P. Wagner, N. Strodthoff, R.-D. Bousseljot, W. Samek, and T. Schaeffter, “PTB-XL, a large publicly available electrocardiography dataset,” *Scientific Data*, vol. 7, no. 1, p. 154, 2020. DOI: 10.1038/s41597-020-0495-6 [Online]. Available: <https://doi.org/10.1038/s41597-020-0495-6>
- [3] Q. Yang, Y. Liu, T. Chen, and Y. Tong, “Federated machine learning: Concept and applications,” *ACM Transactions on Intelligent Systems and Technology*, vol. 10, no. 2, pp. 1–19, 2019. DOI: 10.1145/3298981 [Online]. Available: <https://doi.org/10.1145/3298981>
- [4] J. Konecny, H. B. McMahan, D. Ramage, and P. Richtarik, “Federated optimization: Distributed machine learning for on-device intelligence,” *arXiv preprint arXiv:1610.02527*, 2016. [Online]. Available: <https://arxiv.org/abs/1610.02527>
- [5] F. Hartmann, *Federated learning*, Unpublished thesis, Semantic Scholar, 2018. [Online]. Available: <https://www.semanticscholar.org/paper/Federated-Learning-Hartmann-Rojas/308106a9a8c85241b42bfe447bcf9eab01de3133>
- [6] M. Tariq, M. A. Serhani, F. Sallabi, T. Qayyum, E. Barka, and K. Shaalan, “Trustworthy federated learning: A comprehensive review, architecture, key challenges, and future research prospects,” *IEEE Communications Surveys & Tutorials*, 2024, Advance online publication. [Online]. Available: <https://ieeexplore.ieee.org/document/10623386>
- [7] B. Yuan, S. Ge, and W. Xing, “A federated learning framework for healthcare IoT devices,” *arXiv preprint arXiv:2005.05083*, 2020. [Online]. Available: <https://arxiv.org/abs/2005.05083>
- [8] P. Vepakomma, O. Gupta, T. Swedish, and R. Raskar, “Split learning for health: Distributed deep learning without sharing raw patient data,” *arXiv preprint arXiv:1812.00564*, 2018. [Online]. Available: <https://arxiv.org/abs/1812.00564>
- [9] P. Wagner, N. Strodthoff, R.-D. Bousseljot, W. Samek, and T. Schaeffter, *PTB-XL, a large publicly available electrocardiography dataset (version 1.0.3)*, 2022. DOI: 10.13026/kfzx-aw45 [Online]. Available: <https://doi.org/10.13026/kfzx-aw45>
- [10] A. Paszke et al., “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems*, vol. 32, 2019, pp. 8024–8035. [Online]. Available: <https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html>
- [11] D. J. Beutel, T. Topal, A. Mathur, X. Qiu, T. Parcollet, and N. D. Lane, “Flower: A friendly federated learning research framework,” *arXiv preprint arXiv:2007.14390*, 2020. [Online]. Available: <https://arxiv.org/abs/2007.14390>
- [12] LlamzonAmazon, *GitHub - LlamzonAmazon/asynchronous-fl: A Study of Asynchronous Weight-Updating Federated Learning for Health IoT Devices*, 2026. [Online]. Available: <https://github.com/LlamzonAmazon/asynchronous-fl>